pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

NumPy, Part 1

WEEK 2, DAY 1, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's eighth note

Last week every column question took a loop. This week, one line each, on the four bonds of Days 1 to 4, so every answer checks against last week's: each position's cost and income, each bond's current yield, the top yielder, and the average coupon that matches our cash. Then make me a year of test yield moves, and tell me what QVNE returned this week.

NumPy is the library for this: its array holds a column and calculates on all of it at once.

NumPy, since 2005

Its developers write that it was part of the software behind the discovery of gravitational waves and the first image of a black hole. They list finance and economics among the fields that rely on it.

Source: Harris et al., "Array programming with NumPy", Nature 585, 357-362 (2020)

6.1 A list cannot multiply

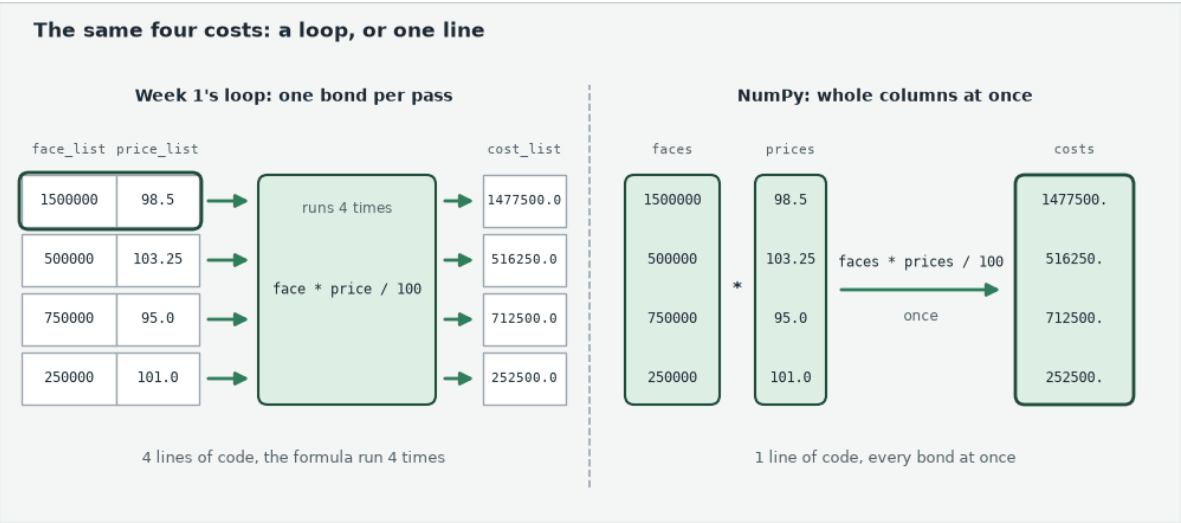
```
ticker_list = ['QVNE', 'DNMR', 'PLWK', 'HLVF']
coupon_list = [5.25, 6.125, 4.75, 7.5]
price_list = [98.5, 103.25, 95.0, 101.0]
face_list = [1500000, 500000, 750000, 250000]
```

```
face_list * price_list / 100
TypeError: can't multiply sequence by non-int of type 'list'
```

```
print(price_list * 2)
morning = [98.5, 103.25]
afternoon = [98.75, 103.0]
print(morning + afternoon)
[98.5, 103.25, 95.0, 101.0, 98.5, 103.25, 95.0, 101.0]
[98.5, 103.25, 98.75, 103.0]
```

A list does no arithmetic: it stops with an error, or worse, answers wrong in silence.

The week 1 way, then one line with NumPy



Week 1: a loop

```
cost_list = []
for face, price in zip(face_list, price_list):
    cost_list.append(face * price / 100)
print(cost_list)
[1477500.0, 516250.0, 712500.0, 252500.0]
```

NumPy: one line

```
import numpy as np
faces = np.array(face_list)
prices = np.array(price_list)
costs = faces * prices / 100
print(costs)
[1477500.  516250.  712500.  252500.]
```

Excel side by side

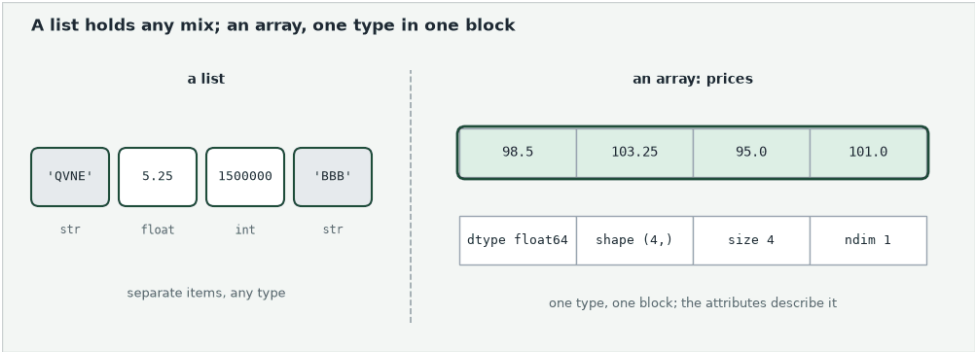
With face in B and price in C, =B2*C2/100 filled down gives the four costs. In Microsoft 365 and Excel 2021, =B2:B5*C2:C5/100 in one cell spills all four: the closer match.

With arrays, * multiplies, value by value.

6.2 One type, one shape

```
print(type(prices))
tickers = np.array(ticker_list)
coupons = np.array([5.25, 6.125, 4.75, 7.5])
print(tickers)
print(coupons)
<class 'numpy.ndarray'>
['QVNE' 'DNMR' 'PLWK' 'HLVF']
[5.25  6.125 4.75  7.5  ]
```

```
print('coupons:', coupons.dtype)
print('faces:', faces.dtype)
print('tickers:', tickers.dtype)
print('size:', prices.size, 'ndim:', prices.ndim, 'shape:', prices.shape)
coupons: float64
faces: int64
tickers: <U4
size: 4 ndim: 1 shape: (4,)
```

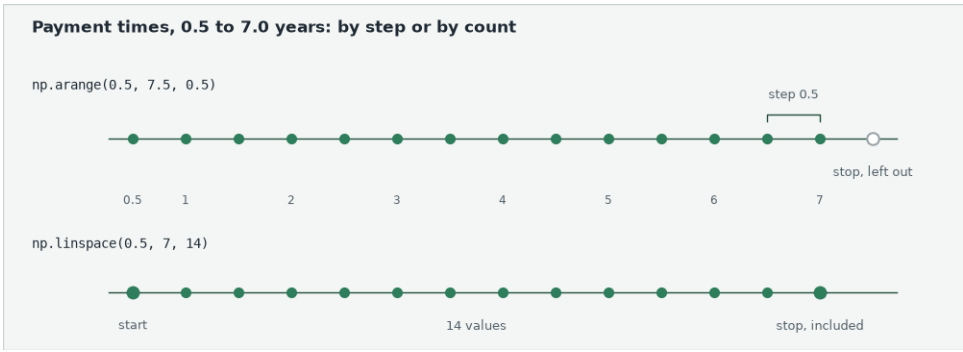


One type is part of what makes an array quick.

float64: a decimal; int64: a whole number; <U4: text of up to 4 characters; (4,): one dimension, four bonds.

Runs of numbers, zeros, and ones

```
print(np.arange(95, 106, 1))
print(np.arange(0.5, 7.5, 0.5))
print(np.linspace(0.5, 7, 14))
[ 95  96  97  98  99 100 101 102 103 104 105]
[0.5 1.  1.5 2.  2.5 3.  3.5 4.  4.5 5.  5.5 6.  6.5 7. ]
[0.5 1.  1.5 2.  2.5 3.  3.5 4.  4.5 5.  5.5 6.  6.5 7. ]
```



Excel side by side

In Microsoft 365 and Excel 2021, =SEQUENCE(11, 1, 95, 1) spills the 11 prices: 11 rows, 1 column, from 95, a step of 1.

Step known: np.arange(). Count known: np.linspace().

Zeros and ones

```
accrued = np.zeros(4)
print(accrued)
redemptions = np.ones(4) * 100
print(redemptions)
[0. 0. 0. 0.]
[100. 100. 100. 100.]
```

6.3 One line for every bond

```
print(prices + 0.5)
print(prices)
[ 99.   103.75  95.5  101.5 ]
[ 98.5  103.25  95.   101.  ]
```

```
print('Total cost:', costs.sum())
print('Annual income:', incomes.sum())
Total cost: 2958750.0
Annual income: 163750.0
```

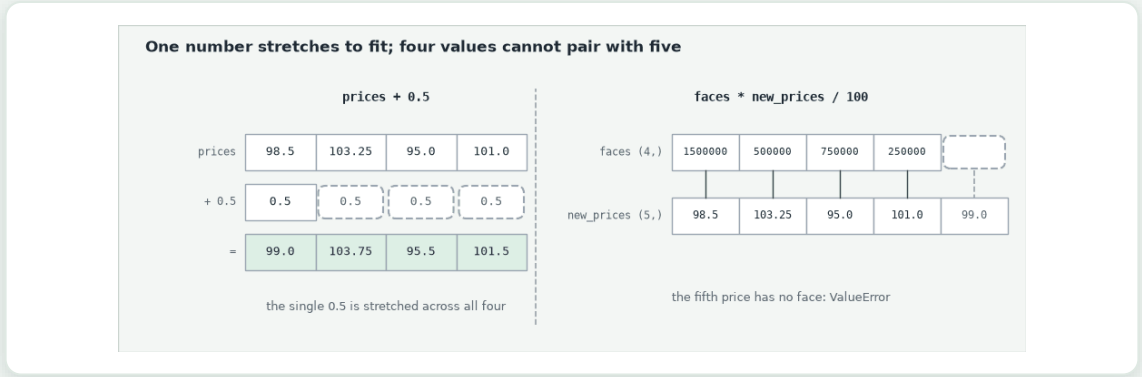
```
incomes = faces * coupons / 100
print(incomes)
current_yields = coupons / prices * 100
print(current_yields)
print(np.round(current_yields, 2))
[78750. 30625. 35625. 18750.]
[5.32994924 5.93220339 5.         7.42574257]
[5.33 5.93 5.   7.43]
```

Question	Python	Excel
Cost of every position	`faces * prices / 100`	`=B2*C2/100 filled down`
Current yield of every bond	`coupons / prices * 100`	`=D2/C2*100 filled down`
Total cost	`costs.sum()`	`=SUM(E2:E5)`
Total cost in one step	`(faces * prices / 100).sum()`	`=SUMPRODUCT(B2:B5, C2:C5)/100`

Decimals, shapes, and a zero price

Compare decimals with np.isclose(), never ==

```
colleague_yields = coupons * 100 / prices
print(current_yields == colleague_yields)
[ True  True  True False]
print(np.isclose(current_yields, colleague_yields))
print(np.allclose(current_yields, colleague_yields))
[ True  True  True  True]
True
```




```
new_prices = np.array([98.5, 103.25, 95.0, 101.0, 99.0])
faces * new_prices / 100
ValueError: operands could not be broadcast together with shapes (4,) (5,)
```

A zero price: a warning, not an error

```
entered_prices = np.array([98.5, 0.0, 95.0, 101.0])
yields_with_zero = coupons / entered_prices * 100
print(yields_with_zero)
[5.32994924        inf 5.         7.42574257]
RuntimeWarning: divide by zero encountered in divide
  yields_with_zero = coupons / entered_prices * 100
```

Predict first: decide, then run it in the notebook

Predict 1

The next cell puts a ticker and its coupon in one array. What happens? A) An error. B) The ticker stays text, the coupon a number. C) Both become text.

```
mixed = np.array(['QVNE', 5.25])
print(mixed)
print(mixed.dtype)
```

Predict 2

The next cell counts from 1 up to 1.3, a step of 0.1, the stop left out. What is the last number? A) 1.2 B) 1.3

```
print(np.arange(1, 1.3, 0.1))
```

Predict 3

The next cell adds up the four yields. What does it print? A) An error. B) About 17.76, the three real yields. C) inf.

```
print(yields_with_zero.sum())
```

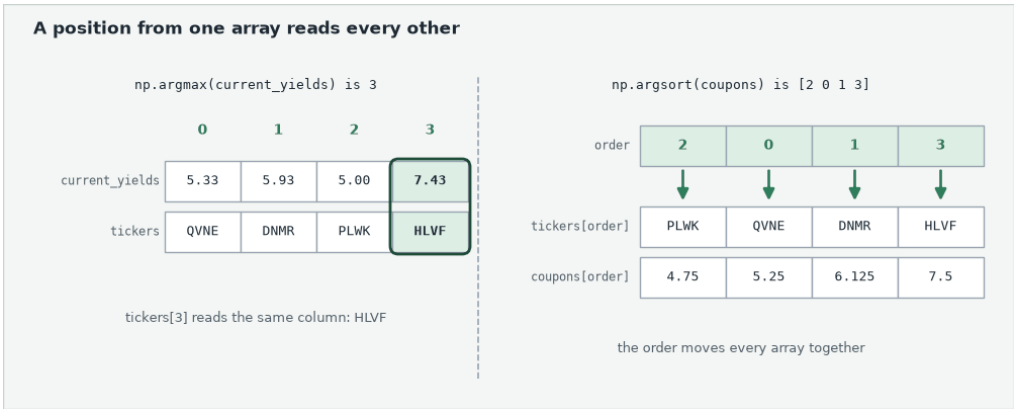
Predict 4

Weight each coupon by its face. Is the weighted average above 5.90625, the same, or below? A) Above. B) The same. C) Below.

```
weighted_average = np.average(coupons, weights=faces)
print(weighted_average)
```

6.4 Summaries, then which bond

```
print('Total face:', np.sum(faces))
print('Average price:', np.mean(prices))
simple_average = np.mean(coupons)
print('Average coupon:', simple_average)
print('Lowest and highest coupon:', np.min(coupons), np.max(coupons))
Total face: 3000000
Average price: 99.4375
Average coupon: 5.90625
Lowest and highest coupon: 4.75 7.5
print(np.cumsum(faces))
[1500000 2000000 2750000 3000000]
```




```
top = np.argmax(current_yields)
print(top, tickers[top])
bottom = np.argmin(current_yields)
print(bottom, tickers[bottom])
3 HLVF
2 PLWK
```

```
print(np.sort(coupons))
order = np.argsort(coupons)
print(order)
print(tickers[order])
[4.75  5.25  6.125 7.5  ]
[2 0 1 3]
['PLWK' 'QVNE' 'DNMR' 'HLVF']
```

Positions keep each bond together. Now the average the note asked for.

Array functions, beside Excel

Python	Excel
`np.sum(faces)`	=SUM(B2:B5)
`np.mean(coupons)`	=AVERAGE(D2:D5)
`np.min(coupons)`	=MIN(D2:D5)
`np.max(coupons)`	=MAX(D2:D5)
`tickers[np.argmax(coupons)]`	=INDEX(A2:A5, MATCH(MAX(D2:D5), D2:D5, 0))
`np.round(current_yields, 2)`	=ROUND(F2, 2) filled down; a half rounds away from zero there
`np.sort(coupons)`	=SORT(D2:D5) (Excel 2021, Microsoft 365)
`tickers[np.argsort(coupons)]`	=SORTBY(A2:A5, D2:D5) (Excel 2021, Microsoft 365)
`np.cumsum(faces)`	=B2 in H2, then =H2+B3 filled down
`np.average(coupons, weights=faces)`	=SUMPRODUCT(D2:D5, B2:B5)/SUM(B2:B5)

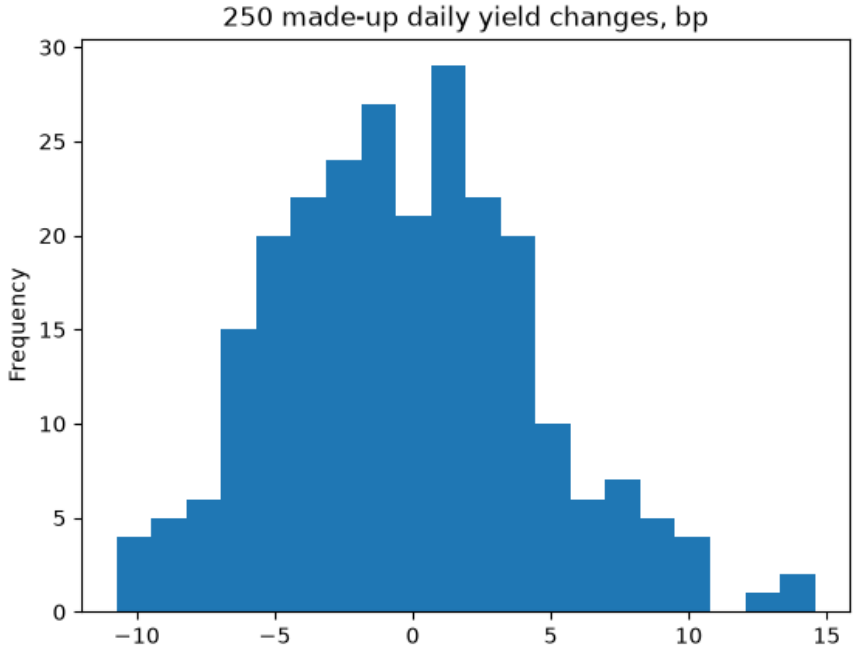
Coupons in D2:D5, face in B2:B5.

How spread out, and a year of made-up yield moves

```
print(np.std(prices))
print(np.std(prices, ddof=1))
3.063775244694036
3.5377429245212264
```

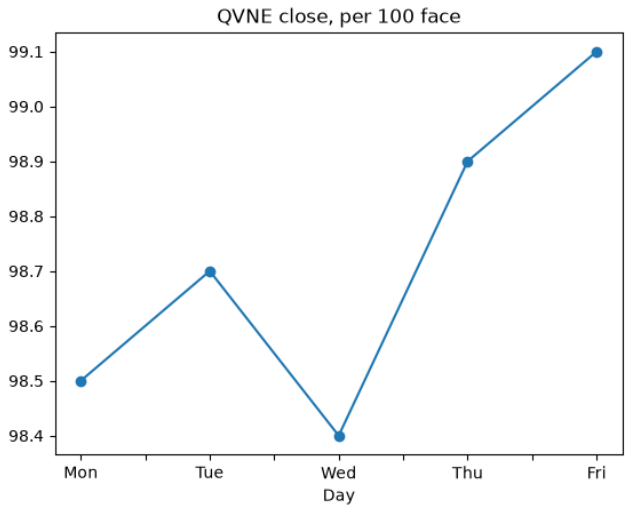
Excel side by side. =STDEV.P(C2:C5) divides by 4, =STDEV.S(C2:C5) by 3.

```
rng = np.random.default_rng(42)
daily_changes_bp = rng.standard_normal(250) * 5
print(np.round(daily_changes_bp[:5], 2))
print(round(daily_changes_bp.mean(), 2), round(daily_changes_bp.std(), 2))
[ 1.52 -5.2   3.75  4.7  -9.76]
-0.24 4.69
```



Same seed, same numbers: a result that can be rerun.

QVNE's week: simple and log returns




```
price_changes = np.diff(qvne_week)
print(np.round(price_changes, 2))
simple_returns = price_changes / qvne_week[:-1]
print(np.round(simple_returns * 100, 4))
[ 0.2 -0.3  0.5  0.2]
[ 0.203  -0.304   0.5081  0.2022]
```

```
log_returns = np.log(qvne_week[1:] / qvne_week[:-1])
print(np.round(log_returns * 100, 4))
[ 0.2028 -0.3044  0.5068  0.202 ]
```

Excel side by side. Closes in B2:B6: the change is =B3-B2 and the simple return =B3/B2-1, filled down. Log: =LN(B3/B2).

```
Simple: 0.006091 sum of days: 0.006094
Log: 0.006073 sum of days: 0.006073
Undo the log: 0.006091
```

Log returns add up across days; simple returns do not.

The note, answered

The note asked	Answer
Cost and income	2,958,750 and 163,750 a year, one line each
Current yields	coupons / prices * 100: 5.00 to 7.43 percent
The top yielder	HLVF, by np.argmax()
A year of test moves	250 seeded draws, the same on every run
QVNE's week	0.609 percent; the daily log returns add up to its log

In NumPy, Part 2, you pick bonds out of an array by a condition, price a bond from its cash flows, and hold the whole book in one array, all with no loop.